

PROJET 4 DATA ANALYST

Réalisez une étude de santé publique avec R ou Python

OBJECTIF DE CE NOTEBOOK

Bienvenue dans l'outil plébiscité par les analystes de données Jupyter.

Il s'agit d'un outil permettant de mixer et d'alterner codes, textes et graphique.

Cet outil est formidable pour plusieurs raisons:

- il permet de tester des lignes de codes au fur et à mesure de votre rédaction, de constater immédiatement le résultat d'une instruction, de la corriger si nécessaire.
- De rédiger du texte pour expliquer l'approche suivie ou les résultats d'une analyse et de le mettre en forme grâce à du code html ou plus simple avec **Markdown**
- d'agrémenter de graphiques

Pour vous aider dans vos premiers pas à l'usage de Jupyter et de Python, nous avons rédigé ce notebook en vous indiquant les instructions à suivre.

Il vous suffit pour cela de saisir le code Python répondant à l'instruction donnée.

Vous verrez de temps à autre le code Python répondant à une instruction donnée mais cela est fait pour vous aider à comprendre la nature du travail qui vous est demandée.

Et garder à l'esprit, qu'il n'y a pas de solution unique pour résoudre un problème et qu'il y a autant de résolutions de problèmes que de développeurs ;)...

Note Jeremy Est ce qu'il faut faire le calcul de la sous nutrition sur les pays qu'on a ? Est ce qu'il faut faire des graphiques ? Rajouter le soja La liste des céréales est difficile à trouver ...

Etape 1 - Importation des librairies et chargement des fichiers

1.1 - Importation des librairies

```
In [2]: #Importation de la librairie Pandas
import pandas as pd
```

1.2 - Chargement des fichiers Excel

```
In [3]: #Importation du fichier population.csv
pop = pd.read_csv('population.csv')

#Importation du fichier dispo_alimentaire.csv
dispo = pd.read_csv('dispo_alimentaire.csv')

#Importation du fichier aide_alimentaire.csv
aide = pd.read_csv('aide_alimentaire.csv')

#Importation du fichier sous_nutrition.csv
sous = pd.read_csv('sous_nutrition.csv')
```

Etape 2 - Analyse exploratoire des fichiers

2.1 - Analyse exploratoire du fichier population

```
In [4]: #Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(pop.shape[0]))
print("Le tableau comporte {} colonne(s)".format(pop.shape[1]))
```

Le tableau comporte 1416 observation(s) ou article(s)
Le tableau comporte 3 colonne(s)

```
In [5]: #Consulter le nombre de colonnes
num_columns = len(pop.columns)
f"Nombre de colonnes : {num_columns}"

#La nature des données dans chacune des colonnes
data_types = pop.dtypes
print("\nNature des données dans chaque colonne :")
print(data_types)

#Le nombre de valeurs présentes dans chacune des colonnes
# Nombre de valeurs présentes dans chaque colonne
non_null_values = pop.count()
print("\nNombre de valeurs non nulles dans chaque colonne :")
print(non_null_values)
```

Nature des données dans chaque colonne :

```
Zone      object
Année     int64
Valeur    float64
dtype: object
```

Nombre de valeurs non nulles dans chaque colonne :

```
Zone      1416
Année     1416
Valeur    1416
dtype: int64
```

```
In [6]: #Affichage les 5 premières lignes de la table
pop.head()
```

```
Out [6]:
```

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

```
In [7]: #Nous allons harmoniser les unités. Pour cela, nous avons décidé de multiplier la colonne valeur par 1000
pop['Valeur'] = pop['Valeur'] * 1000
pop.head()
```

```
Out [7]:
```

	Zone	Année	Valeur
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

```
In [8]: #changement du nom de la colonne Valeur par Population
pop.rename(columns={'Valeur': 'Population'}, inplace=True)
print(pop.columns)
```

```
Index(['Zone', 'Année', 'Population'], dtype='object')
```

```
In [9]: #Affichage les 5 premières lignes de la table pour voir les modifications
pop.head()
```

Out [9]:

	Zone	Année	Population
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

```
In [10]: # Renommer manuellement les colonnes pour les rendre en minuscule, sans a
pop.rename(columns={
    'Zone': 'zone',
    'Année': 'annee',
    'Population': 'population'
}, inplace=True)

# Vérification des nouveaux noms de colonnes
print(pop.columns)
```

```
Index(['zone', 'annee', 'population'], dtype='object')
```

2.2 - Analyse exploratoire du fichier disponibilité alimentaire

```
In [11]: #Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(dispo.
print("Le tableau comporte {} colonne(s)".format(dispo.shape[1]))
```

```
Le tableau comporte 15605 observation(s) ou article(s)
Le tableau comporte 18 colonne(s)
```

```
In [12]: #Consulter le nombre de colonnes
num_columns = len(dispo.columns)
f"Nombre de colonnes : {num_columns}"
```

```
Out[12]: 'Nombre de colonnes : 18'
```

```
In [13]: #Affichage les 5 premières lignes de la table
dispo.head()
```

Out [13]:

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	(k
0	Afghanistan	Abats Comestible	animale	NaN	NaN	5.0	
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.0	
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.0	
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.0	
4	Afghanistan	Bananes	vegetale	NaN	NaN	4.0	

In [14]: `#remplacement des NaN dans le dataset par des 0`
`dispo.fillna(0, inplace=True)`

In [15]: `#multiplication de toutes les lignes contenant des milliers de tonnes en`
`colonnes_a_convertir = [`
 `'Aliments pour animaux', 'Autres Utilisations', 'Disponibilité intéri`
 `'Exportations – Quantité', 'Importations – Quantité', 'Nourriture',`
 `'Pertes', 'Production', 'Semences', 'Traitement', 'Variation de stock`
`]`
`dispo[colonnes_a_convertir] = dispo[colonnes_a_convertir] * 1_000_000`

In [16]: `#Affichage les 5 premières lignes de la table`
`dispo.head()`

Out [16]:

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/personne/jour)	(k
0	Afghanistan	Abats Comestible	animale	0.0	0.0	5.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.0	
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.0	
4	Afghanistan	Bananes	vegetale	0.0	0.0	4.0	

In [17]: `dispo.rename(columns={`
 `'Zone': 'zone',`
 `'Produit': 'produit',`
 `'Origine': 'origine',`
 `'Aliments pour animaux': 'aliments_pour_animaux',`
 `'Autres Utilisations': 'autres_utilisations',`
 `'Disponibilité alimentaire (Kcal/personne/jour)': 'disponibilite_alim`

```
'Disponibilité alimentaire en quantité (kg/personne/an)': 'disponibil
'Disponibilité de matière grasse en quantité (g/personne/jour)': 'dis
'Disponibilité de protéines en quantité (g/personne/jour)': 'disponib
'Disponibilité intérieure': 'disponibilite_interieure',
'Exportations - Quantité': 'exportations_quantite',
'Importations - Quantité': 'importations_quantite',
'Nourriture': 'nourriture',
'Pertes': 'pertes',
'Production': 'production',
'Semences': 'semences',
'Traitement': 'traitement',
'Variation de stock': 'variation_de_stock'
}, inplace=True)
```

2.3 - Analyse exploratoire du fichier aide alimentaire

```
In [18]: #Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(aide.s
print("Le tableau comporte {} colonne(s)".format(aide.shape[1]))
```

Le tableau comporte 1475 observation(s) ou article(s)
Le tableau comporte 4 colonne(s)

```
In [19]: #Consulter le nombre de colonnes
num_columns = len(aide.columns)
f"Nombre de colonnes : {num_columns}"
```

Out[19]: 'Nombre de colonnes : 4'

```
In [20]: #Affichage les 5 premières lignes de la table
aide.head()
```

Out[20]:

	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

```
In [21]: #changement du nom de la colonne Pays bénéficiaire par Zone
aide.rename(columns={'pays_beneficiaire': 'zone'}, inplace=True)
```

```
In [22]: #Multiplication de la colonne Aide_alimentaire qui contient des tonnes pa
aide['Valeur'] = aide['Valeur'] * 1000
```

```
In [23]: #Affichage les 5 premières lignes de la table
aide.head()
```

Out [23]:

	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

```
In [24]: aide.rename(columns={
    'Pays bénéficiaire': 'pays_beneficiaire',
    'Année': 'annee',
    'Produit': 'produit',
    'Valeur': 'valeur'
}, inplace=True)
```

2.3 - Analyse exploratoire du fichier sous nutrition

```
In [25]: #Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou article(s)".format(sous.s
print("Le tableau comporte {} colonne(s)".format(sous.shape[1]))
```

Le tableau comporte 1218 observation(s) ou article(s)
Le tableau comporte 3 colonne(s)

```
In [26]: #Consulter le nombre de colonnes
num_columns = len(sous.columns)
f"Nombre de colonnes : {num_columns}"
```

Out [26]: 'Nombre de colonnes : 3'

```
In [27]: #Afficher les 5 premières lignes de la table
sous.head()
```

Out [27]:

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

```
In [30]: # Supprimer les lignes où la colonne 'Valeur' contient des valeurs non nu
sous = sous[pd.to_numeric(sous['Valeur'], errors='coerce').notna()]

#Conversion de la colonne sous nutrition en numérique
sous['Valeur'] = pd.to_numeric(sous['Valeur'])
```

```
In [31]: #Conversion de la colonne (avec l'argument errors=coerce qui permet de co
#Puis remplacement des NaN en 0
```

```
sous['Valeur'] = pd.to_numeric(sous['Valeur'], errors='coerce')
sous['Valeur'] = sous['Valeur'].fillna(0)
```

```
In [32]: #changement du nom de la colonne Valeur par sous_nutrition
sous.rename(columns={'Zone': 'zone', 'Année': 'annee', 'Valeur': 'valeur'})
print(sous.columns)

sous.rename(columns={'valeur': 'sous_nutrition'}, inplace=True)
print(sous.columns)
```

```
Index(['zone', 'annee', 'valeur'], dtype='object')
Index(['zone', 'annee', 'sous_nutrition'], dtype='object')
```

```
In [33]: #Multiplication de la colonne sous_nutrition par 1000000
sous['sous_nutrition'] = sous['sous_nutrition'] * 1_000_000
```

```
In [34]: #Afficher les 5 premières lignes de la table
sous.head()
```

```
Out[34]:
```

	zone	annee	sous_nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

3.1 - Proportion de personnes en sous nutrition

```
In [36]: pop.head(1)
```

```
Out[36]:
```

	zone	annee	population
0	Afghanistan	2013	32269589.0

```
In [37]: sous.head(1)
```

```
Out[37]:
```

	zone	annee	sous_nutrition
0	Afghanistan	2012-2014	8600000.0

```
In [42]: # Extraire l'année médiane pour chaque plage dans la table sous
sous['annee'] = sous['annee'].str.split('-').apply(lambda x: int((int(x[0]) + int(x[1])) / 2))

# Filtrer les données pour l'année médiane égale à 2017
sous_2017 = sous[sous['annee'] == 2017]

# Filtrer les données dans la table pop pour l'année 2017
pop_2017 = pop[pop['annee'] == 2017]

# Effectuer la jointure sur les colonnes 'zone' et 'annee'
```



```
resultat_jointure = pd.merge(pop_2017, sous_2017, on=['zone', 'annee'], h
# Afficher les 5 premières lignes du résultat
resultat_jointure.head()
```

Out [42]:

	zone	annee	population	sous_nutrition
0	Afghanistan	2017	36296113.0	10500000.0
1	Afrique du Sud	2017	57009756.0	3100000.0
2	Albanie	2017	2884169.0	100000.0
3	Algérie	2017	41389189.0	1300000.0
4	Angola	2017	29816766.0	5800000.0

In [55]:

```
# Étape 2 : Filtrer pour l'année 2017
pop_2017 = pop[pop['annee'] == 2017]
sous_2017 = sous[sous['annee'] == 2017]
```

In [57]:

```
sous_2017.head()
```

Out [57]:

	zone	annee	sous_nutrition
4	Afghanistan	2017	10500000.0
10	Afrique du Sud	2017	3100000.0
16	Albanie	2017	100000.0
22	Algérie	2017	1300000.0
40	Angola	2017	5800000.0

In [58]:

```
# Étape 3 : Jointure entre 'pop' et 'sous'
resultat_jointure = pd.merge(pop_2017, sous_2017, on=['zone', 'annee'], h
```

In [59]:

```
resultat_jointure.head()
```

Out [59]:

	zone	annee	population	sous_nutrition
0	Afghanistan	2017	36296113.0	10500000.0
1	Afrique du Sud	2017	57009756.0	3100000.0
2	Albanie	2017	2884169.0	100000.0
3	Algérie	2017	41389189.0	1300000.0
4	Angola	2017	29816766.0	5800000.0

In [62]:

```
# Calcul de la population mondiale en 2017
population_mondiale_2017 = pop_2017['population'].sum()

# Calcul du nombre de personnes en état de sous-nutrition en 2017
personnes_sous_nutrition_2017 = sous_2017['sous_nutrition'].sum()

# Calcul du taux de sous-nutrition en 2017
taux_sous_nutrition_2017 = personnes_sous_nutrition_2017 / population_mon
```

```

taux_sous_nutrition_2017_percent = round(taux_sous_nutrition_2017 * 100,

# Affichage des résultats
print(f"En 2017, on a dénombré {int(personnes_sous_nutrition_2017)} perso
print(f"Le taux de sous-nutrition en 2017 est de {taux_sous_nutrition_201

```

En 2017, on a dénombré 535700000 personnes en état de sous-nutrition à tra
vers le monde.

Le taux de sous-nutrition en 2017 est de 0.07097118203283073 soit 7.1 %

3.2 - Nombre théorique de personne qui pourrait être nourries

```
In [ ]: #Combien mange en moyenne un être humain ? Source => 2400 Kcal
```

```
In [72]: calories_min = 2100
```

```
In [63]: dispo.head(10)
```

```
Out [63]:
```

	zone	produit	origine	aliments_pour_animaux	autres_utilisations	di
0	Afghanistan	Abats Comestible	animale	0.0	0.0	
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	
3	Afghanistan	Ananas	vegetale	0.0	0.0	
4	Afghanistan	Bananes	vegetale	0.0	0.0	
5	Afghanistan	Beurre, Ghee	animale	0.0	0.0	
6	Afghanistan	Bière	vegetale	0.0	0.0	
7	Afghanistan	Blé	vegetale	0.0	0.0	
8	Afghanistan	Boissons Alcooliques	vegetale	0.0	0.0	
9	Afghanistan	Café	vegetale	0.0	0.0	

```

In [64]: # Regrouper par la colonne 'zone' et calculer la somme pour les autres co
dispo_grouped = dispo.groupby('zone', as_index=False).sum()

# Afficher le résultat
dispo_grouped.head()

```

Out [64]:

	zone	produit	origine
0	Afghanistan	Abats ComestibleAgrumes, AutresAliments pour e...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
1	Afrique du Sud	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
2	Albanie	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
3	Algérie	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
4	Allemagne	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...

In [66]:

```
#On commence par faire une jointure entre le data frame population et Dis
# Réaliser la jointure entre les tables dispo et pop sur la colonne 'zone'
dispo_avec_population = pd.merge(dispo_grouped, pop_2017, on='zone', how=
# Afficher les premières lignes du résultat
dispo_avec_population.head()
```

Out [66]:

	zone	produit	origine
0	Afghanistan	Abats ComestibleAgrumes, AutresAliments pour e...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
1	Afrique du Sud	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
2	Albanie	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
3	Algérie	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
4	Allemagne	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...

```
In [67]: #Affichage du nouveau dataframe  
dispo_avec_population.head()
```

```
Out [67]:
```

	zone	produit	origine
0	Afghanistan	Abats ComestibleAgrumes, AutresAliments pour e...	animalevegetalevegetalevegetalevegetaleanimale...
1	Afrique du Sud	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetaleanimale...
2	Albanie	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetaleanimale...
3	Algérie	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetaleanimale...
4	Allemagne	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetaleanimale...

```
In [70]: #Création de la colonne dispo_kcal avec calcul des kcal disponibles mondi  
dispo_avec_population['dispo_kcal'] = dispo_avec_population['disponibilit  
dispo_avec_population.head()
```

Out [70]:

	zone	produit	origine
0	Afghanistan	Abats ComestibleAgrumes, AutresAliments pour e...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
1	Afrique du Sud	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
2	Albanie	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
3	Algérie	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
4	Allemagne	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...

5 rows × 21 columns

In [99]: `calorie_dispo = int(dispo_avec_population["dispo_kcal"].sum())`In [100...]: `#Calcul du nombre d'humains pouvant être nourris
personnes_nourries = int(calorie_dispo/calories_min)`In [101...]: `pop_mond_2017 = dispo_avec_population['population'].sum()`In [102...]: `"La somme des valeurs de la colonne 'population' est :", pop_mond_2017`Out[102...]: `("La somme des valeurs de la colonne 'population' est :",
np.float64(7291900830.0))`In [138...]: `# Assurez-vous que le ratio est converti en float natif de Python
ratio = round(float(personnes_nourries / pop_mond_2017 * 100), 2)

Affichage du résultat
f"En théorie, nous devrions être capable de nourrir {personnes_nourries}"`Out[138...]: `'En théorie, nous devrions être capable de nourrir 9961421251 personnes
dans le monde, soit 136.61 % de la population mondiale.'`

3.3 - Nombre théorique de personne qui pourrait être nourrie avec les produits végétaux

In [104...]: `#Transfert des données avec les végétaux dans un nouveau dataframe
dispo_vegetale = dispo[dispo['origine'] == 'vegetale']`

In [105... `dispo_vegetale.head()`

Out [105...

	zone	produit	origine	aliments_pour_animaux	autres_utilisations	disp
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	
3	Afghanistan	Ananas	vegetale	0.0	0.0	
4	Afghanistan	Bananes	vegetale	0.0	0.0	
6	Afghanistan	Bière	vegetale	0.0	0.0	

In [106... `dispo_vegetale_grouped = dispo_vegetale.groupby("zone", as_index=False).s`

In [107... `dispo_vegetale_grouped.head()`

Out [107...

	zone	aliments_pour_animaux	autres_utilisations	disponibilite_alimentaire
0	Afghanistan	6.450000e+08	4.150000e+08	
1	Afrique du Sud	5.122000e+09	7.610000e+08	
2	Albanie	5.590000e+08	1.720000e+08	
3	Algérie	4.352000e+09	1.706000e+09	
4	Allemagne	2.812200e+10	6.842000e+09	

In [131...

```
#La jointure :
dispo_merged = pd.merge(dispo_vegetale_grouped, pop_2017, on='zone', how=
# Afficher les premières lignes du résultat
dispo_merged.head()
```

Out [131...

	zone	aliments_pour_animaux	autres_utilisations	disponibilite_alimentaire
0	Afghanistan	6.450000e+08	4.150000e+08	
1	Afrique du Sud	5.122000e+09	7.610000e+08	
2	Albanie	5.590000e+08	1.720000e+08	
3	Algérie	4.352000e+09	1.706000e+09	
4	Allemagne	2.812200e+10	6.842000e+09	

In [108...

```
#Calcul du nombre de kcal disponible pour les végétaux
kcal_vegetaux = dispo_vegetale_grouped['disponibilite_alimentaire_kcal_p
```

In [149... `dispo_merged['dispo_kcal_vegetaux'] = dispo_merged['disponibilite_alimen`

In [150... `dispo_merged.head()`

	zone	aliments_pour_animaux	autres_utilisations	disponibilite_alimentaire
0	Afghanistan	6.450000e+08	4.150000e+08	
1	Afrique du Sud	5.122000e+09	7.610000e+08	
2	Albanie	5.590000e+08	1.720000e+08	
3	Algérie	4.352000e+09	1.706000e+09	
4	Allemagne	2.812200e+10	6.842000e+09	

```
In [151... calories_vegetales_dispo = int(dispo_merged["dispo_kcal_vegeteaux"].sum())
```

```
In [152... #Calcul du nombre d'humains pouvant être nourris
personnes_nourries_vegeteaux = int(calories_vegetales_dispo/calories_min)
```

```
In [153... pop_monde_2017 = dispo_merged['population'].sum()
```

```
In [154... "La somme des valeurs de la colonne 'population' est :", pop_monde_2017
```

```
Out[154... ("La somme des valeurs de la colonne 'population' est :",
np.float64(7291900830.0))
```

```
In [156... ratio_v = round(float(personnes_nourries_vegeteaux / pop_monde_2017 * 100

# Affichage du résultat
f"Nous pouvons nourrir à base de végétaux {personnes_nourries_vegeteaux}
```

```
Out[156... 'Nous pouvons nourrir à base de végétaux 8219411529 personnes dans le m
onde, soit 112.72 % de la population mondiale.'
```

3.4 - Utilisation de la disponibilité intérieure

```
In [158... dispo_avec_population.head()
```

Out [158...

	zone	produit	origine
0	Afghanistan	Abats ComestibleAgrumes, AutresAliments pour e...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
1	Afrique du Sud	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
2	Albanie	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
3	Algérie	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
4	Allemagne	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...

5 rows × 21 columns

In [159... *#Calcul de la disponibilité totale*
 disponibilite_totale = dispo_avec_population['disponibilite_interieure'].

In [111... *#création d'une boucle for pour afficher les différentes valeurs en fonct*

In [160... total_aliments_pour_animaux = dispo_avec_population['aliments_pour_animau

In [161... total_perte = dispo_avec_population['pertes'].sum()

In [162... total_nourriture = dispo_avec_population['nourriture'].sum()

In [164... part_animaux = round((total_aliments_pour_animaux / disponibilite_totale)
 print(f"La proportion d'aliment pour animaux par rapport à la disponibili

La proportion d'aliment pour animaux par rapport à la disponibilité intéri
 eure est de 13.23 %

In [269... part_perte = round((total_perte / disponibilite_totale) * 100, 2)
 print(f"La proportion de pertes par rapport à la disponibilité intérieure

La proportion de pertes par rapport à la disponibilité intérieure est de
 4.65 %

In [270... part_nourriture = round((total_nourriture / disponibilite_totale) * 100,
 print(f"La proportion de nourriture par rapport à la disponibilité intéri

La proportion de nourriture par rapport à la disponibilité intérieure est
 de 49.37 %

3.5 - Utilisation des céréales

```
In [171... #Création d'une liste avec toutes les variables
cereales_names = [ "Blé", "Riz (Eq Blanchi)", "Orge", "Maïs", "Seigle", "
```

```
In [172... #Création d'un dataframe avec les informations uniquement pour ces céréales
cereales_df = dispo[dispo['produit'].isin(cereales_names)]
```

```
In [174... dispo_cereales = cereales_df.groupby("zone", as_index=False).sum(numeric_
```

```
In [176... dispo_cereales.head()
```

```
Out[176...
      zone  aliments_pour_animaux  autres_utilisations  disponibilite_alimentaire
0  Afghanistan      5.600000e+08      0.000000e+00
1  Afrique du Sud      4.824000e+09      0.000000e+00
2    Albanie      4.110000e+08      1.340000e+08
3    Algérie      4.337000e+09      8.200000e+08
4  Allemagne      2.661500e+10      1.013000e+09
```

```
In [177... disponibilite_cereales_interieure = dispo_cereales['disponibilite_interie
```

```
In [178... total_aliments_pour_animaux_cereales = dispo_cereales['aliments_pour_anim
```

```
In [179... #Affichage de la proportion d'alimentation animale

proportion_animale = round((total_aliments_pour_animaux_cereales / dispon
print(f"La proportion d'alimentation animale par rapport à la disponibili
```

La proportion d'alimentation animale par rapport à la disponibilité intérieure des céréales est de 36.29 %

3.6 - Pays avec la proportion de personnes sous-alimentée la plus forte en 2017

```
In [183... resultat_jointure.head()
```

```
Out[183...
      zone  annee  population  sous_nutrition
0  Afghanistan  2017  36296113.0  10500000.0
1  Afrique du Sud  2017  57009756.0  3100000.0
2    Albanie  2017  2884169.0  100000.0
3    Algérie  2017  41389189.0  1300000.0
4    Angola  2017  29816766.0  5800000.0
```

```
In [187... #Création de la colonne proportion par pays
resultat_jointure['proportion_sous_alimentation'] = (resultat_jointure['s
resultat_jointure.head()
```

```
Out[187...      zone  annee  population  sous_nutrition  proportion_sous_alimentation
0  Afghanistan    2017   36296113.0      10500000.0          28.928718
1  Afrique du Sud    2017   57009756.0       3100000.0          5.437666
2    Albanie    2017    2884169.0        100000.0          3.467203
3    Algérie    2017   41389189.0       1300000.0          3.140917
4    Angola    2017   29816766.0       5800000.0         19.452143
```

```
In [188... #affichage après trie des 10 pires pays
top_10_pays = resultat_jointure.sort_values(
    by='proportion_sous_alimentation', ascending=False
).head(10)
```

```
In [189... top_10_pays.head(10)
```

```
Out[189...      zone  annee  population  sous_nutrition  proportion_sous_alimentation
32    Haïti    2017   10982366.0       5300000.0          48.259182
67  République populaire démocratique de Corée    2017   25429825.0      12000000.0          47.188685
45  Madagascar    2017   25570512.0      10500000.0          41.062924
44    Libéria    2017    4702226.0       1800000.0          38.279742
42    Lesotho    2017    2091534.0        800000.0          38.249438
76    Tchad    2017   15016753.0       5700000.0          37.957606
69    Rwanda    2017   11980961.0       4200000.0          35.055619
53  Mozambique    2017   28649018.0       9400000.0          32.810898
78  Timor-Leste    2017    1243258.0        400000.0          32.173531
0    Afghanistan    2017   36296113.0      10500000.0          28.928718
```

3.7 - Pays qui ont le plus bénéficié d'aide alimentaire depuis 2013

```
In [190... aide.head()
```

Out [190...

	pays_beneficiaire	annee	produit	valeur
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

In [192...

```
# Regrouper par pays (zone) et calculer la somme des colonnes numériques
aide_grouped = aide.groupby('pays_beneficiaire', as_index=False).sum()

# Afficher le résultat
aide_grouped.head()
```

Out [192...

	pays_beneficiaire	annee	produit	valeur
0	Afghanistan	32216	Autres non-céréalesAutres non-céréalesBlé et F...	185452000
1	Algérie	78561	Autres non-céréalesAutres non-céréalesAutres n...	81114000
2	Angola	8054	Autres non-céréalesCéréalesMélanges et prépara...	5014000
3	Bangladesh	50347	Blé et FarinBlé et FarinBlé et FarinCéréalesCé...	348188000
4	Bhoutan	24170	CéréalesHuiles végétalesHuiles végétalesHuiles...	2666000

In [193...

```
#affichage après trie des 10 pays qui ont bénéficié le plus de l'aide ali
#affichage après trie des 10 pires pays
top_10_pays_aide = aide_grouped.sort_values(
    by='valeur', ascending=False
).head(10)
```

In [194...

```
top_10_pays_aide.head(10)
```

Out [194...

	pays_beneficiaire	annee	produit	valeur
50	République arabe syrienne	76543	Autres non-céréalesAutres non-céréalesAutres n...	1858943000
75	Éthiopie	66460	Autres non-céréalesAutres non-céréalesAutres n...	1381294000
70	Yémen	76553	Autres non-céréalesAutres non-céréalesAutres n...	1206484000
61	Soudan du Sud	44309	Autres non-céréalesAutres non-céréalesAutres n...	695248000
60	Soudan	48335	Autres non-céréalesAutres non-céréalesBlé et F...	669784000
30	Kenya	54376	Autres non-céréalesAutres non-céréalesBlé et F...	552836000
3	Bangladesh	50347	Blé et FarinBlé et FarinBlé et FarinCéréalesCé...	348188000
59	Somalie	54377	Autres non-céréalesAutres non-céréalesAutres n...	292678000
53	République démocratique du Congo	50350	Autres non-céréalesAutres non-céréalesAutres n...	288502000
43	Niger	62448	Autres non-céréalesAutres non-céréalesAutres n...	276344000

3.8 - Evolution des 5 pays qui ont le plus bénéficiés de l'aide alimentaire entre 2013 et 2016

```
In [ ]: #Création d'un dataframe avec la zone, l'année et l'aide alimentaire puis

# Sélection des colonnes nécessaires
aide_grouped = aide[['pays_beneficiaire', 'annee', 'valeur']]

# Groupement par pays bénéficiaire et année, en sommant les valeurs
aide_grouped = aide_grouped.groupby(['pays_beneficiaire', 'annee'], as_in

# Renommer les colonnes pour plus de clarté
aide_grouped.rename(columns={'pays_beneficiaire': 'zone', 'valeur': 'aide

# Affichage du résultat
print(aide_grouped)
```

	zone	annee	aide_alimentaire
0	Afghanistan	2013	128238000
1	Afghanistan	2014	57214000
2	Algérie	2013	35234000
3	Algérie	2014	18980000
4	Algérie	2015	17424000
...	...	...	...
223	Égypte	2013	1122000
224	Équateur	2013	1362000
225	Éthiopie	2013	591404000
226	Éthiopie	2014	586624000
227	Éthiopie	2015	203266000

[228 rows x 3 columns]

```
In [196... #Création d'une liste contenant les 5 pays qui ont le plus bénéficiées de
# Calcul de la somme de l'aide alimentaire par pays
top_aide_pays = aide_grouped.groupby('zone')['aide_alimentaire'].sum().re
# Tri des pays par ordre décroissant de l'aide alimentaire reçue
top_aide_pays = top_aide_pays.sort_values(by='aide_alimentaire', ascendi
# Extraction des 5 pays ayant reçu le plus d'aide
top_5_pays = top_aide_pays.head(5)
# Création d'une liste des noms des pays
liste_top_5_pays = top_5_pays['zone'].tolist()
# Affichage du résultat
print(liste_top_5_pays)
['République arabe syrienne', 'Éthiopie', 'Yémen', 'Soudan du Sud', 'Souda
n']
```

```
In [197... #On filtre sur le dataframe avec notre liste
# Liste des pays à filtrer
pays_a_filtrer = ['République arabe syrienne', 'Éthiopie', 'Yémen', 'Soud
# Filtrer le DataFrame pour ne garder que les pays dans la liste
aide_filtree = aide_grouped[aide_grouped['zone'].isin(pays_a_filtrer)]
# Affichage du résultat
print(aide_filtree)
```

		zone	annee	aide_alimentaire
157	République arabe syrienne		2013	563566000
158	République arabe syrienne		2014	651870000
159	République arabe syrienne		2015	524949000
160	République arabe syrienne		2016	118558000
189		Soudan	2013	330230000
190		Soudan	2014	321904000
191		Soudan	2015	176500000
192		Soudan du Sud	2013	196330000
193		Soudan du Sud	2014	450610000
194		Soudan du Sud	2015	483080000
214		Yémen	2013	264764000
215		Yémen	2014	103840000
216		Yémen	2015	372306000
217		Yémen	2016	465574000
225		Éthiopie	2013	591404000
226		Éthiopie	2014	586624000
227		Éthiopie	2015	203266000

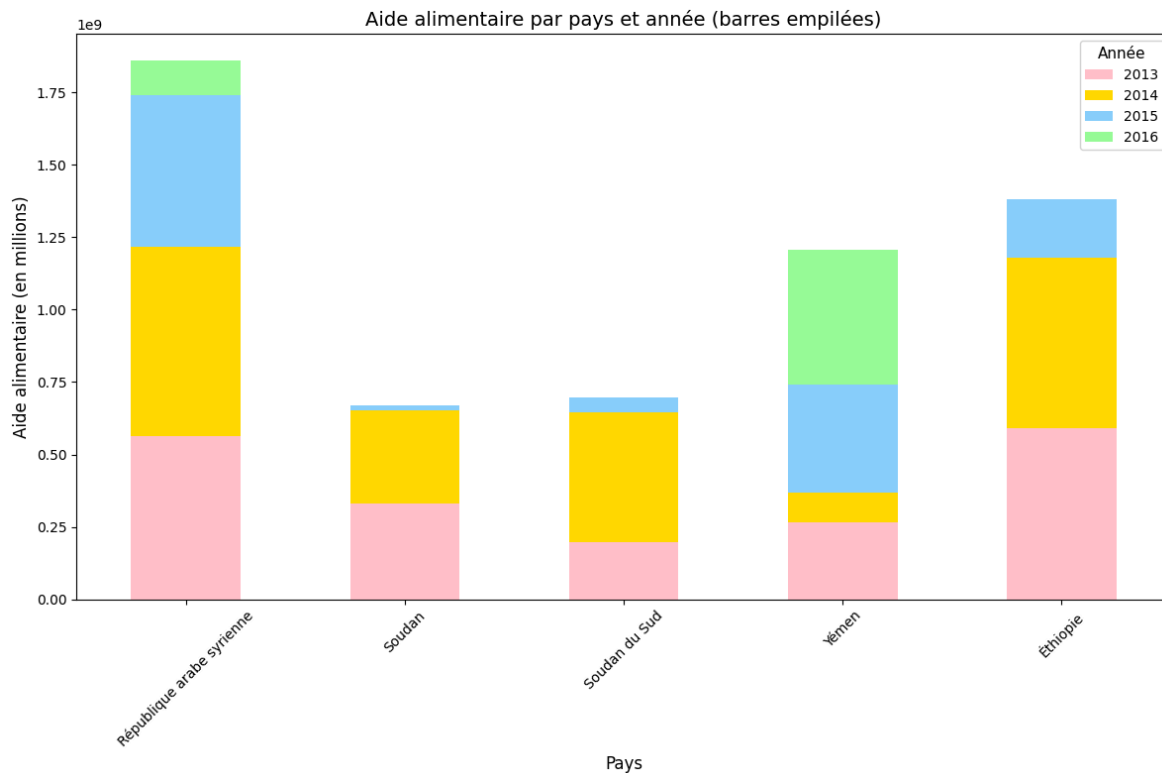
```
In [203... import matplotlib.pyplot as plt

# Transformation des données pour l'affichage (pivot pour le stacked bar
aide_filtree_pivot = aide_filtree.pivot(index='zone', columns='annee', va

# Création du graphique
colors = ['#FFC0CB', '#FFD700', '#87CEFA', '#98FB98', '#DDA0DD'] # Coule
aide_filtree_pivot.plot(
    kind='bar',
    stacked=True,
    figsize=(12, 8),
    color=colors
)

# Paramètres du graphique
plt.title("Aide alimentaire par pays et année (barres empilées)", fontsize=12)
plt.xlabel("Pays", fontsize=12)
plt.ylabel("Aide alimentaire (en millions)", fontsize=12)
plt.xticks(rotation=45, fontsize=10)
plt.legend(title="Année", fontsize=10, title_fontsize=11)
plt.tight_layout()

# Afficher le graphique
plt.show()
```



3.9 - Pays avec le moins de disponibilité par habitant

In [204...] `dispo.head(1)`

Out [204...]

	zone	produit	origine	aliments_pour_animaux	autres_utilisations	dis
0	Afghanistan	Abats Comestible	animale	0.0	0.0	

In [207...]

```
dispo_avec_pop = pd.merge(dispo_grouped, pop_2017, on='zone', how='inner')

# Afficher les premières lignes du résultat
dispo_avec_pop.head()
```

Out [207...

	zone	produit	origine
0	Afghanistan	Abats ComestibleAgrumes, AutresAliments pour e...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
1	Afrique du Sud	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
2	Albanie	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
3	Algérie	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...
4	Allemagne	Abats ComestibleAgrumes, AutresAlcool, non Com...	animalevegetalevegetalevegetalevegetalevegetaleanimale...

In [209...

```
#Affichage des 10 pays qui ont le moins de dispo alimentaire par personne

# Tri des pays par disponibilité alimentaire par personne (croissant)
top_10_lowest_dispo = dispo_avec_pop.sort_values(by='disponibilite_alimen

# Afficher les résultats
top_10_lowest_dispo[['zone', 'disponibilite_alimentaire_kcal_par_personne
```

Out [209...

	zone	disponibilite_alimentaire_kcal_par_personne_par_jour
127	République centrafricaine	1879.0
164	Zambie	1924.0
91	Madagascar	2056.0
0	Afghanistan	2087.0
65	Haïti	2089.0
132	République populaire démocratique de Corée	2093.0
150	Tchad	2109.0
165	Zimbabwe	2113.0
114	Ouganda	2126.0
152	Timor-Leste	2129.0

In [210...

```
import matplotlib.pyplot as plt

# Trier les pays par disponibilité alimentaire par personne (croissant)
```

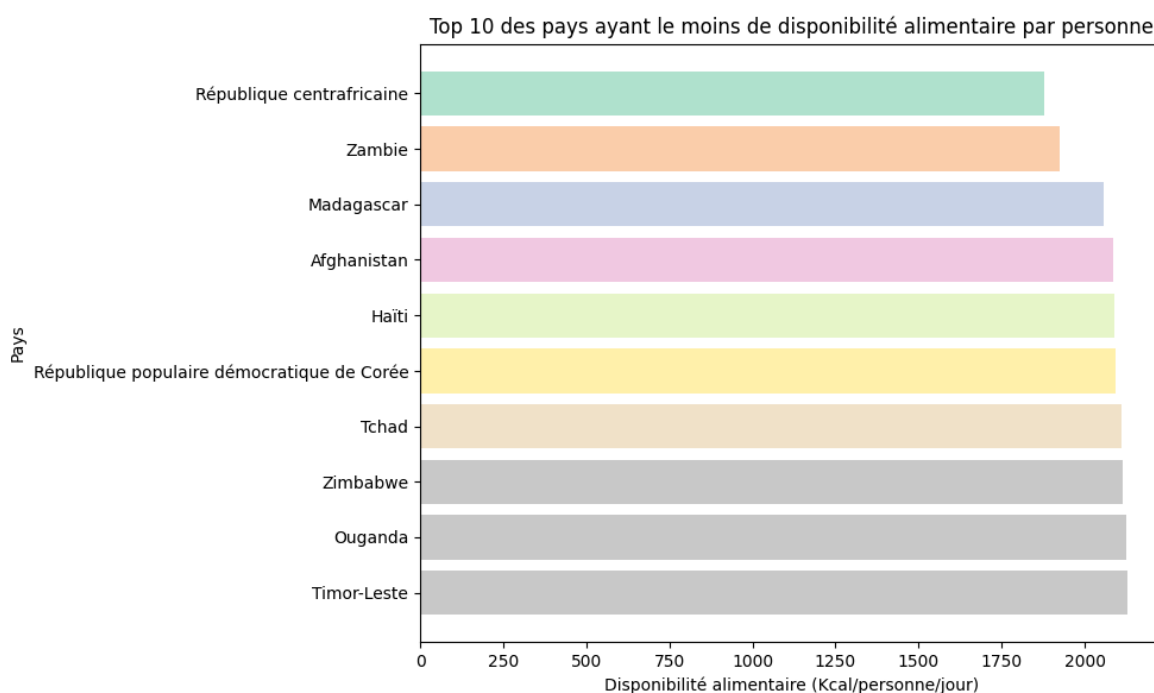


```

top_10_lowest_dispo = dispo_avec_pop.sort_values(by='disponibilite_alimen

# Création du graphique en pyramide inversée
plt.figure(figsize=(10, 6))
plt.barh(
    top_10_lowest_dispo['zone'],
    top_10_lowest_dispo['disponibilite_alimentaire_kcal_par_personne_par_
    color=plt.cm.Pastel2(range(len(top_10_lowest_dispo)))
)
plt.xlabel('Disponibilité alimentaire (Kcal/personne/jour)')
plt.ylabel('Pays')
plt.title('Top 10 des pays ayant le moins de disponibilité alimentaire pa
plt.gca().invert_yaxis() # Inversion de l'axe des pays pour obtenir une
plt.tight_layout()
plt.show()

```



3.10 - Pays avec le plus de disponibilité par habitant

```

In [211... #Affichage des 10 pays qui ont le plus de dispo alimentaire par personne
# données par disponibilité alimentaire par personne de manière décroissa
top_10_dispo_par_personne = dispo_avec_pop.sort_values(by='disponibilite_

# Afficher les 10 premiers pays
top_10_dispo_par_personne[['zone', 'disponibilite_alimentaire_kcal_par_pe

```

Out [211...

	zone	disponibilite_alimentaire_kcal_par_personne_par_jour
11	Autriche	3770.0
16	Belgique	3737.0
157	Turquie	3708.0
169	États-Unis d'Amérique	3682.0
74	Israël	3610.0
72	Irlande	3602.0
75	Italie	3578.0
89	Luxembourg	3540.0
166	Égypte	3518.0
4	Allemagne	3503.0

In [213...

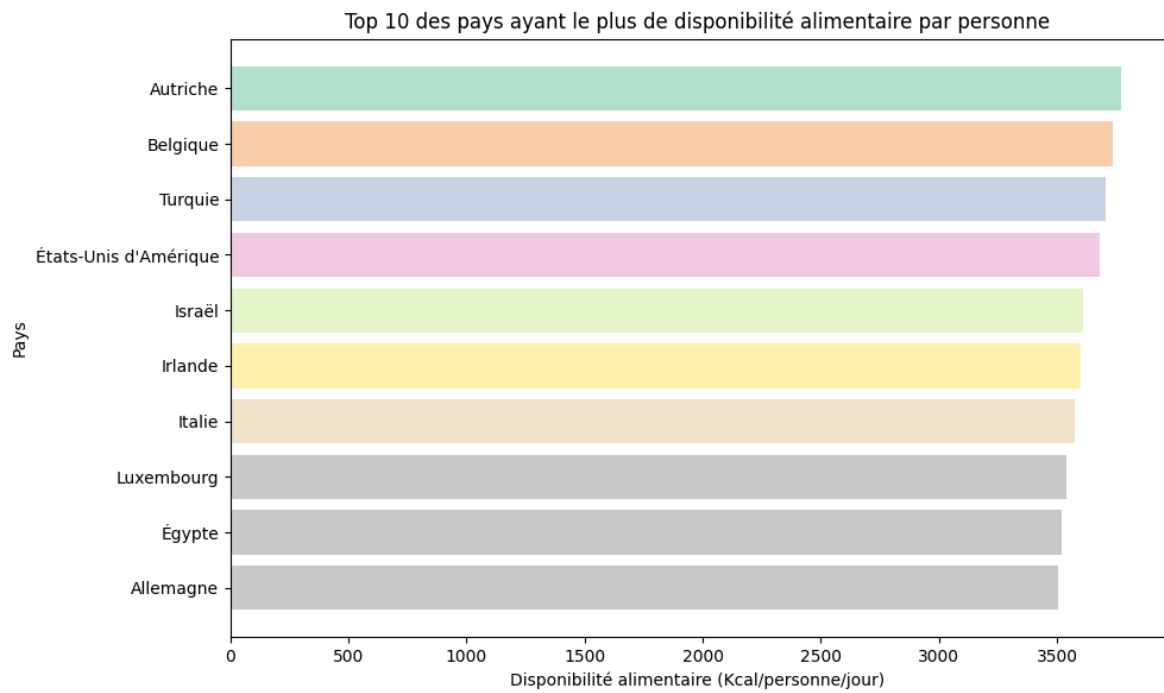
```

import matplotlib.pyplot as plt

# Trier les pays par disponibilité alimentaire par personne (décroissant)
top_10_highest_dispo = dispo_avec_pop.sort_values(
    by='disponibilite_alimentaire_kcal_par_personne_par_jour', ascending=
).head(10)

# Création du graphique en pyramide inversée pour les pays avec la plus h
plt.figure(figsize=(10, 6))
plt.barh(
    top_10_highest_dispo['zone'],
    top_10_highest_dispo['disponibilite_alimentaire_kcal_par_personne_par
    color=plt.cm.Pastel2(range(len(top_10_highest_dispo)))
)
plt.xlabel('Disponibilité alimentaire (Kcal/personne/jour)')
plt.ylabel('Pays')
plt.title('Top 10 des pays ayant le plus de disponibilité alimentaire par
plt.gca().invert_yaxis() # Inversion de l'axe des pays pour obtenir une
plt.tight_layout()
plt.show()

```



3.11 - Exemple de la Thaïlande pour le Manioc

```
In [214... #création d'un dataframe avec uniquement la Thaïlande
pop_thaïlande = pop[pop['zone'] == 'Thaïlande']
```

```
In [215... dispo_thaïlande = pd.merge(dispo, pop_thaïlande, on='zone', how='inner')
```

```
In [231... dispo_thaïlande.head(50)
```

Out [231...

	zone	produit	origine	aliments_pour_animaux	autres_utilisations	d
0	Thaïlande	Abats Comestible	animale	0.0	0.0	
1	Thaïlande	Abats Comestible	animale	0.0	0.0	
2	Thaïlande	Abats Comestible	animale	0.0	0.0	
3	Thaïlande	Abats Comestible	animale	0.0	0.0	
4	Thaïlande	Abats Comestible	animale	0.0	0.0	
5	Thaïlande	Abats Comestible	animale	0.0	0.0	
6	Thaïlande	Agrumes, Autres	vegetale	0.0	0.0	
7	Thaïlande	Agrumes, Autres	vegetale	0.0	0.0	
8	Thaïlande	Agrumes, Autres	vegetale	0.0	0.0	
9	Thaïlande	Agrumes, Autres	vegetale	0.0	0.0	
10	Thaïlande	Agrumes, Autres	vegetale	0.0	0.0	
11	Thaïlande	Agrumes, Autres	vegetale	0.0	0.0	
12	Thaïlande	Alcool, non Comestible	vegetale	0.0	358000000.0	
13	Thaïlande	Alcool, non Comestible	vegetale	0.0	358000000.0	
14	Thaïlande	Alcool, non Comestible	vegetale	0.0	358000000.0	
15	Thaïlande	Alcool, non Comestible	vegetale	0.0	358000000.0	
16	Thaïlande	Alcool, non Comestible	vegetale	0.0	358000000.0	
17	Thaïlande	Alcool, non Comestible	vegetale	0.0	358000000.0	
18	Thaïlande	Aliments pour enfants	vegetale	0.0	0.0	
19	Thaïlande	Aliments pour enfants	vegetale	0.0	0.0	
20	Thaïlande	Aliments pour enfants	vegetale	0.0	0.0	
21	Thaïlande	Aliments pour enfants	vegetale	0.0	0.0	

	zone	produit	origine	aliments_pour_animaux	autres_utilisations	d
22	Thaïlande	Aliments pour enfants	vegetale	0.0	0.0	
23	Thaïlande	Aliments pour enfants	vegetale	0.0	0.0	
24	Thaïlande	Ananas	vegetale	0.0	0.0	
25	Thaïlande	Ananas	vegetale	0.0	0.0	
26	Thaïlande	Ananas	vegetale	0.0	0.0	
27	Thaïlande	Ananas	vegetale	0.0	0.0	
28	Thaïlande	Ananas	vegetale	0.0	0.0	
29	Thaïlande	Ananas	vegetale	0.0	0.0	
30	Thaïlande	Animaux Aquatiques Autre	animale	0.0	0.0	
31	Thaïlande	Animaux Aquatiques Autre	animale	0.0	0.0	
32	Thaïlande	Animaux Aquatiques Autre	animale	0.0	0.0	
33	Thaïlande	Animaux Aquatiques Autre	animale	0.0	0.0	
34	Thaïlande	Animaux Aquatiques Autre	animale	0.0	0.0	
35	Thaïlande	Animaux Aquatiques Autre	animale	0.0	0.0	
36	Thaïlande	Arachides Decortiquees	vegetale	0.0	0.0	
37	Thaïlande	Arachides Decortiquees	vegetale	0.0	0.0	
38	Thaïlande	Arachides Decortiquees	vegetale	0.0	0.0	
39	Thaïlande	Arachides Decortiquees	vegetale	0.0	0.0	
40	Thaïlande	Arachides Decortiquees	vegetale	0.0	0.0	
41	Thaïlande	Arachides Decortiquees	vegetale	0.0	0.0	
42	Thaïlande	Avoine	vegetale	0.0	0.0	
43	Thaïlande	Avoine	vegetale	0.0	0.0	
44	Thaïlande	Avoine	vegetale	0.0	0.0	

	zone	produit	origine	aliments_pour_animaux	autres_utilisations	d
45	Thaïlande	Avoine	vegetale	0.0	0.0	
46	Thaïlande	Avoine	vegetale	0.0	0.0	
47	Thaïlande	Avoine	vegetale	0.0	0.0	
48	Thaïlande	Bananes	vegetale	0.0	0.0	
49	Thaïlande	Bananes	vegetale	0.0	0.0	

In [217...] `sous_thaïlande = pd.merge(sous_2017, pop_thaïlande, on=('zone', 'annee'),`

In [218...] `sous_thaïlande.head()`

Out[218...]

	zone	annee	sous_nutrition	population
0	Thaïlande	2017	6200000.0	69209810.0

In [220...]

```
# Calculer le pourcentage de sous-nutrition
sous_thaïlande['sous_nutrition_pourcentage'] = (sous_thaïlande['sous_nutr
# Afficher le résultat formaté en pourcentage
sous_thaïlande['sous_nutrition_pourcentage'] = sous_thaïlande['sous_nutri
sous_thaïlande
```

Out[220...]

	zone	annee	sous_nutrition	population	sous_nutrition_pourcentage
0	Thaïlande	2017	6200000.0	69209810.0	8.96%

In [221...]

```
import matplotlib.pyplot as plt

# Préparation des données
sous_nutrition = sous_thaïlande['sous_nutrition'].values[0]
population_totale = sous_thaïlande['population'].values[0]
autres = population_totale - sous_nutrition

# Création des labels et des tailles
labels = ['Sous-nutrition', 'Autres']
sizes = [sous_nutrition, autres]

# Couleurs en pastel
colors = ['#ff9999', '#66b3ff']

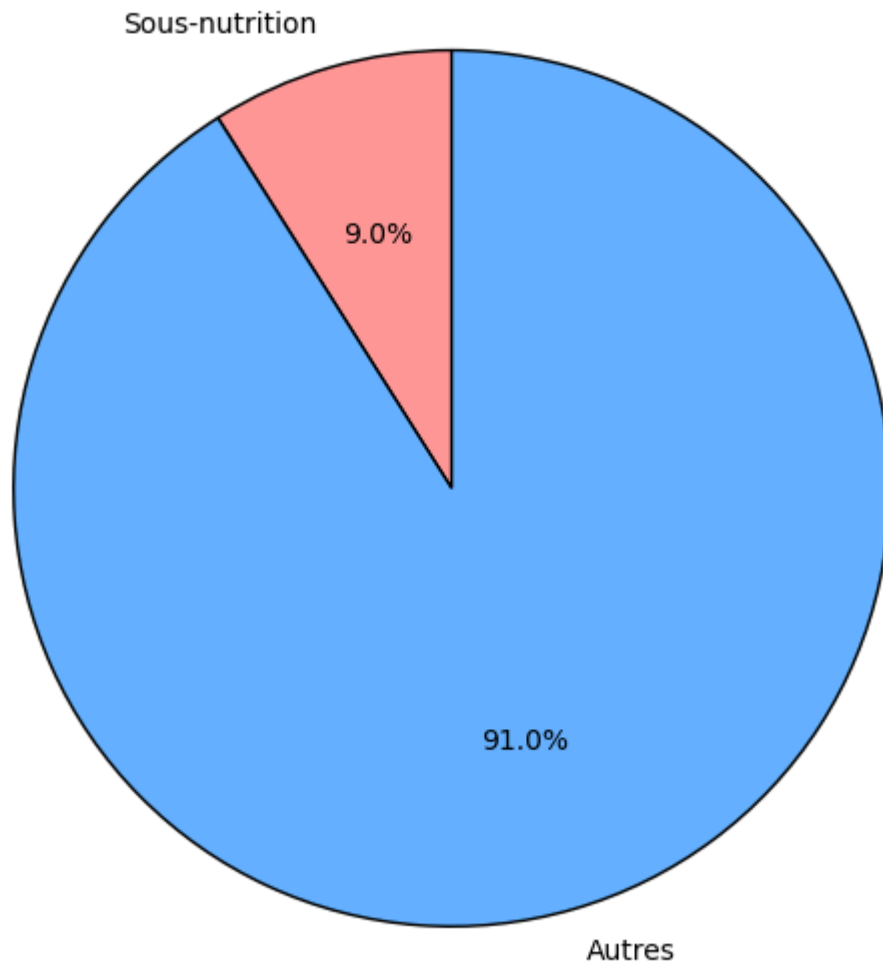
# Création du pie chart
plt.figure(figsize=(6, 6))
plt.pie(
    sizes,
    labels=labels,
    autopct='%1.1f%%',
    startangle=90,
    colors=colors,
    wedgeprops={'edgecolor': 'black'}
)

# Titre
```

```
plt.title('Répartition de la population thaïlandaise (2017)')

# Afficher le graphique
plt.tight_layout()
plt.show()
```

Répartition de la population thaïlandaise (2017)



In [129...] *# On calcule la proportion exportée en fonction de la proportion*

In [234...] *# Filtrer les lignes où le produit est "manioc" et l'année est 2017*
dispo_thaïlande_manioc_2017 = dispo_thaïlande[(dispo_thaïlande['produit']

Afficher les résultats filtrés
dispo_thaïlande_manioc_2017

Out[234...]

	zone	produit	origine	aliments_pour_animaux	autres_utilisations	dispo
304	Thaïlande	Manioc	vegetale	1,800,000,000.00	2,081,000,000.00	

In [224...] *# Calcul de la proportion d'exportation du manioc*
proportion_manioc_thaïlande = round(
 (dispo_thaïlande_manioc_2017["exportations_quantite"].iloc[0] / dispo
 2
)

```
f"La proportion d'exportation de manioc en Thaïlande en 2017 est de : {pr
```

```
Out[224...] "La proportion d'exportation de manioc en Thaïlande en 2017 est de : 83.41 %"
```

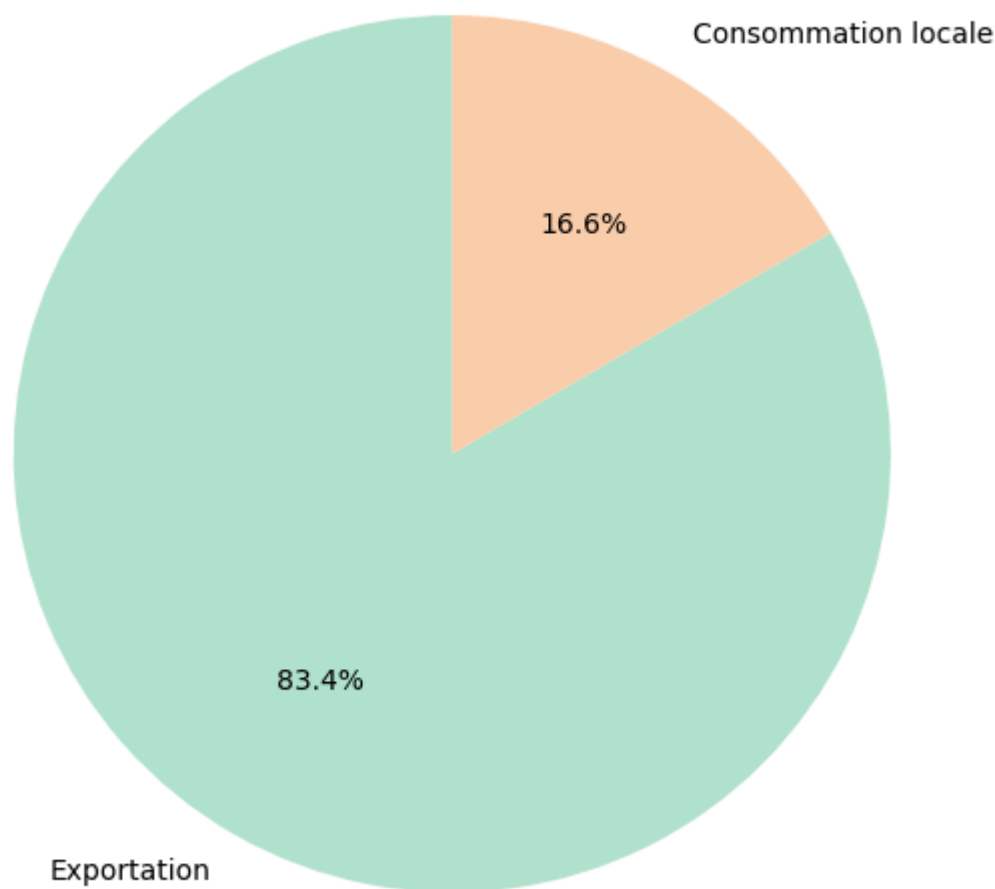
```
In [225...] import matplotlib.pyplot as plt

# Données pour le graphique
exportation = dispo_thaïlande_manioc_2017["exportations_quantite"].iloc[0]
consommation_locale = dispo_thaïlande_manioc_2017["production"].iloc[0] -

# Création des labels et des valeurs
labels = ['Exportation', 'Consommation locale']
values = [exportation, consommation_locale]

# Création du pie chart
plt.figure(figsize=(8, 6))
plt.pie(
    values,
    labels=labels,
    autopct='%1.1f%%',
    startangle=90,
    colors=plt.cm.Pastel2.colors
)
plt.title("Proportion d'exportation du manioc en Thaïlande en 2017")
plt.tight_layout()
plt.show()
```


Proportion d'exportation du manioc en Thaïlande en 2017



Etape 6 - Analyse complémentaires

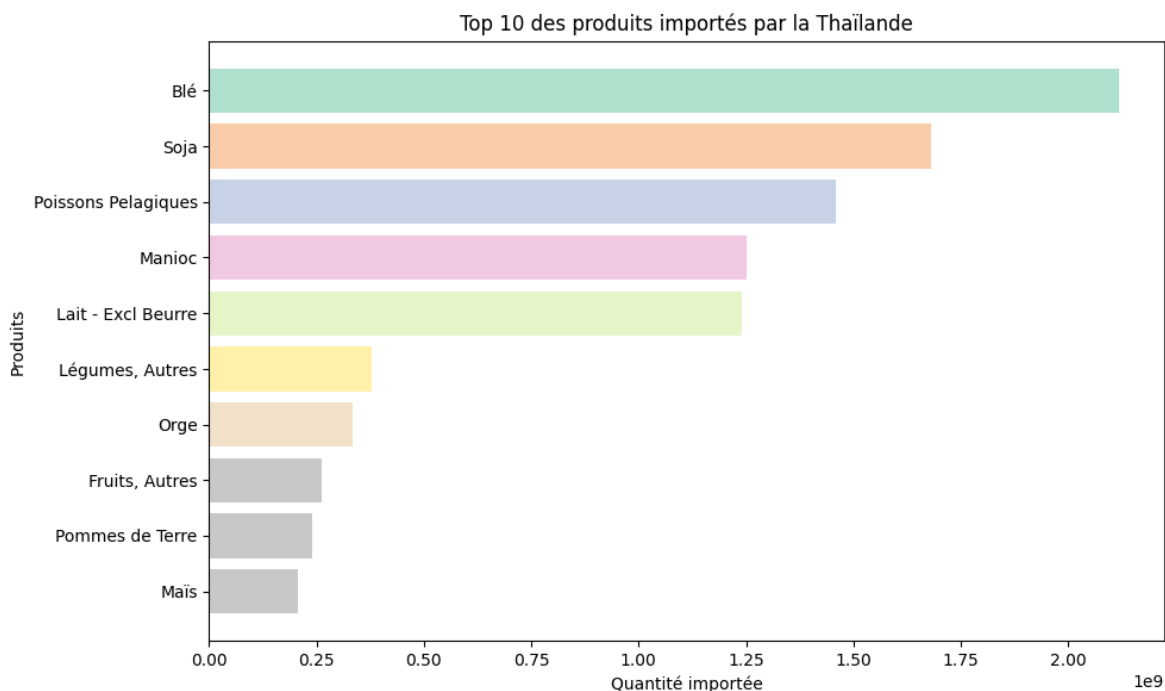
```
In [130... #Rajouter en dessous toutes les analyses complémentaires suite à la demande  
#"et toutes les infos que tu trouverais utiles pour mettre en relief les  
#le plus en difficulté au niveau alimentaire"
```

```
In [ ] : # creuser au niveau de la thaïlande / export du manioc + aides de pays //
```

```
In [233... # Filtrer les données pour la Thaïlande  
thaïlande_importations = dispo[dispo['zone'] == 'Thaïlande']  
  
# Trier les produits par quantité d'importation (décroissant) et sélection  
top_10_importations = thaïlande_importations.sort_values(by='importations'  
  
# Afficher les résultats  
print(top_10_importations[['produit', 'importations_quantite']])  
  
# Optionnel : Afficher les résultats dans un bar chart  
import matplotlib.pyplot as plt  
  
plt.figure(figsize=(10, 6))
```

```
plt.barh(
    top_10_importations['produit'],
    top_10_importations['importations_quantite'],
    color=plt.cm.Pastel2(range(len(top_10_importations)))
)
plt.xlabel('Quantité importée')
plt.ylabel('Produits')
plt.title('Top 10 des produits importés par la Thaïlande')
plt.gca().invert_yaxis() # Inversion de l'axe pour un meilleur affichage
plt.tight_layout()
plt.show()
```

	produit	importations_quantite
13770	Blé	2,118,000,000.00
13839	Soja	1,682,000,000.00
13831	Poissons Pelagiques	1,460,000,000.00
13809	Manioc	1,250,000,000.00
13806	Lait – Excl Beurre	1,241,000,000.00
13807	Légumes, Autres	378,000,000.00
13820	Orge	335,000,000.00
13783	Fruits, Autres	261,000,000.00
13834	Pommes de Terre	240,000,000.00
13810	Maïs	206,000,000.00



In [235...] `aide_grouped.head()`

Out [235...]

	zone	annee	aide_alimentaire
0	Afghanistan	2013	128238000
1	Afghanistan	2014	57214000
2	Algérie	2013	35234000
3	Algérie	2014	18980000
4	Algérie	2015	17424000

In [238... *# Création d'un nouveau DataFrame à partir de la table aide_grouped sans aide_without_year* = aide.drop(columns=['annee'])

In [239... aide_without_year.head()

Out [239...

	pays_beneficiaire	produit	valeur
0	Afghanistan	Autres non-céréales	682000
1	Afghanistan	Autres non-céréales	335000
2	Afghanistan	Blé et Farin	39224000
3	Afghanistan	Blé et Farin	15160000
4	Afghanistan	Céréales	40504000

In [240... *# Regrouper les lignes ayant les mêmes pays et sommer les valeurs corresp*
 aide_grouped_by_country = aide_without_year.groupby("pays_beneficiaire").
 aide_grouped_by_country.head()

Out [240...

	pays_beneficiaire	produit	valeur
0	Afghanistan	Autres non-céréalesAutres non-céréalesBlé et F...	185452000
1	Algérie	Autres non-céréalesAutres non-céréalesAutres n...	81114000
2	Angola	Autres non-céréalesCéréalesMélanges et prépara...	5014000
3	Bangladesh	Blé et FarinBlé et FarinBlé et FarinCéréalesCé...	348188000
4	Bhoutan	CéréalesHuiles végétalesHuiles végétalesHuiles...	2666000

In [243... *# Renommer la colonne 'pays_beneficiaire' en 'zone'*
 aide_grouped_by_country.rename(columns={'pays_beneficiaire': 'zone'}, inp

Jointure avec la table pop_2017
 aide_population_merged = pd.merge(aide_grouped_by_country, pop_2017, on=''

Affichage des premières lignes du dataframe résultant
 aide_population_merged.head()

Out [243...

	zone	produit	valeur	annee	population
0	Afghanistan	Autres non-céréales non-céréalesBlé et F...	185452000	2017	36,296,113.00
1	Algérie	Autres non-céréales non-céréalesAutres n...	81114000	2017	41,389,189.00
2	Angola	Autres non- céréalesCéréalesMélanges et prépara...	5014000	2017	29,816,766.00
3	Bangladesh	Blé et FarinBlé et FarinBlé et FarinCéréalesCé...	348188000	2017	159,685,424.00
4	Bhoutan	CéréalesHuiles végétalesHuiles végétalesHuiles...	2666000	2017	745,563.00

In [245...

```
# Création d'une nouvelle colonne 'aide_par_personne' en kg par personne
aide_population_merged['aide_par_personne'] = aide_population_merged['val
aide_population_merged[['zone', 'valeur', 'population', 'aide_par_personn
```

Out [245...

	zone	valeur	population	aide_par_personne
0	Afghanistan	185452000	36,296,113.00	5.11
1	Algérie	81114000	41,389,189.00	1.96
2	Angola	5014000	29,816,766.00	0.17
3	Bangladesh	348188000	159,685,424.00	2.18
4	Bhoutan	2666000	745,563.00	3.58

In [247...

```
# 2. Extraction des pays recevant le plus d'aide alimentaire (correction
top_aide_alimentaire = aide.groupby('pays_beneficiaire')['valeur'].sum().
```

In [248...

```
# 1. Extraction des pays avec les taux de sous-nutrition les plus élevés
top_sous_nutrition = sous.groupby('zone')['sous_nutrition'].sum().sort_va

# 2. Extraction des pays recevant le plus d'aide alimentaire (colonne cor
top_aide_alimentaire = aide.groupby('pays_beneficiaire')['valeur'].sum().

# 3. Comparaison des deux listes
top_sous_nutrition_pays = set(top_sous_nutrition.index)
top_aide_alimentaire_pays = set(top_aide_alimentaire.index)

# 4. Pays présents dans les deux listes (corrélacion)
pays_communs = top_sous_nutrition_pays.intersection(top_aide_alimentaire_

# 5. Pays en sous-nutrition mais peu aidés
pays_sous_nutrition_non_aides = top_sous_nutrition_pays - top_aide_alimen

# 6. Pays très aidés mais avec peu de sous-nutrition
pays_aides_non_sous_nutrition = top_aide_alimentaire_pays - top_sous_nutr

# 📊 Résultats
print("✅ Pays à la fois très aidés et très touchés par la sous-nutrition
```

```
print("⚠ Pays très touchés par la sous-nutrition mais peu aidés :", pays_aidés)
print("? Pays très aidés mais avec moins de sous-nutrition :", pays_aidés)
```

✓ Pays à la fois très aidés et très touchés par la sous-nutrition : {'Bangladesh', 'Kenya', 'Éthiopie'}

⚠ Pays très touchés par la sous-nutrition mais peu aidés : {'République populaire démocratique de Corée', 'Inde', 'République-Unie de Tanzanie', 'Philippines', 'Indonésie', 'Pakistan', 'Nigéria'}

? Pays très aidés mais avec moins de sous-nutrition : {'Yémen', 'République démocratique du Congo', 'Soudan du Sud', 'Somalie', 'Niger', 'Soudan', 'République arabe syrienne'}

```
In [256... from matplotlib_venn import venn2
import matplotlib.pyplot as plt

# Création d'un diagramme de Venn
plt.figure(figsize=(8, 6))
venn2([top_sous_nutrition_pays, top_aide_alimentaire_pays],
      set_labels=('Sous-nutrition élevée', 'Aide alimentaire élevée'))
plt.title("Comparaison des pays en sous-nutrition et recevant de l'aide alimentaire")
plt.show()
```

Comparaison des pays en sous-nutrition et recevant de l'aide alimentaire



```
In [259... import matplotlib.pyplot as plt

# Conversion des données en format numérique si nécessaire (suppression de
dispo_avec_pop['disponibilite_alimentaire_kcal_par_personne_par_jour'] =

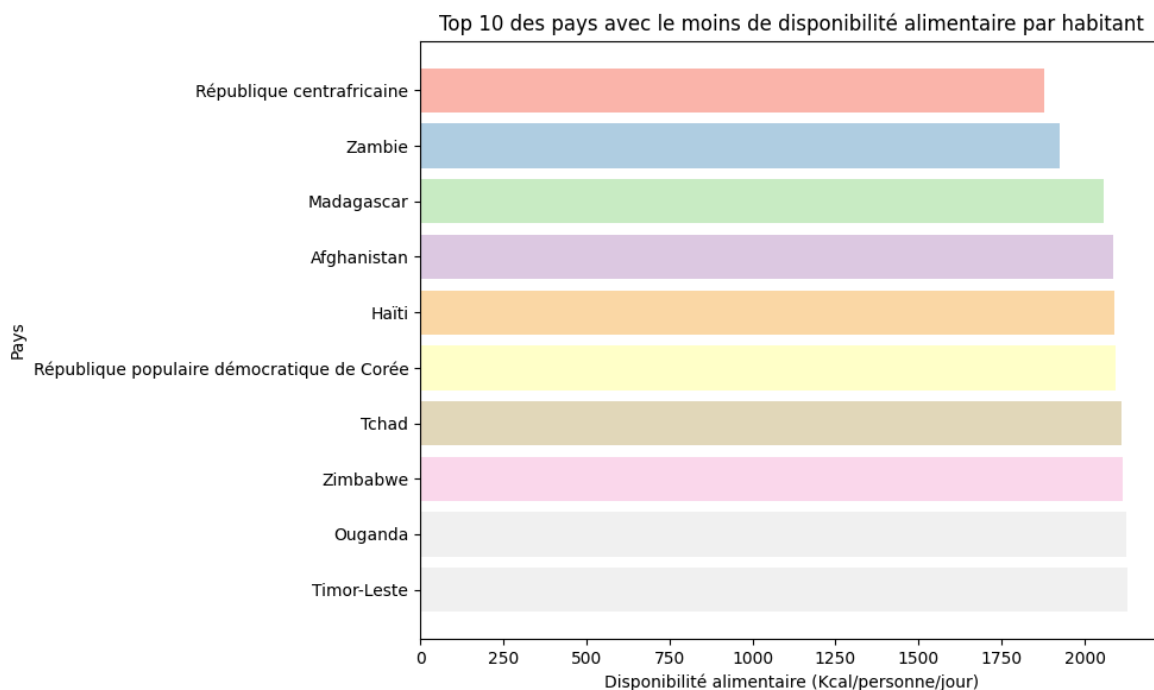
# Trier les pays par disponibilité alimentaire croissante et sélectionner
top_10_lowest = dispo_avec_pop.sort_values(by='disponibilite_alimentaire_

# Création du Bar Chart
plt.figure(figsize=(10, 6))
plt.barh(
    top_10_lowest['zone'],
    top_10_lowest['disponibilite_alimentaire_kcal_par_personne_par_jour']
```

```

    color=plt.cm.Pastel1(range(len(top_10_lowest)))
)
plt.xlabel('Disponibilité alimentaire (Kcal/personne/jour)')
plt.ylabel('Pays')
plt.title('Top 10 des pays avec le moins de disponibilité alimentaire par
plt.gca().invert_yaxis() # Inversion pour effet pyramide inversée
plt.tight_layout()
plt.show()

```



```

In [261... import matplotlib.pyplot as plt
import squarify

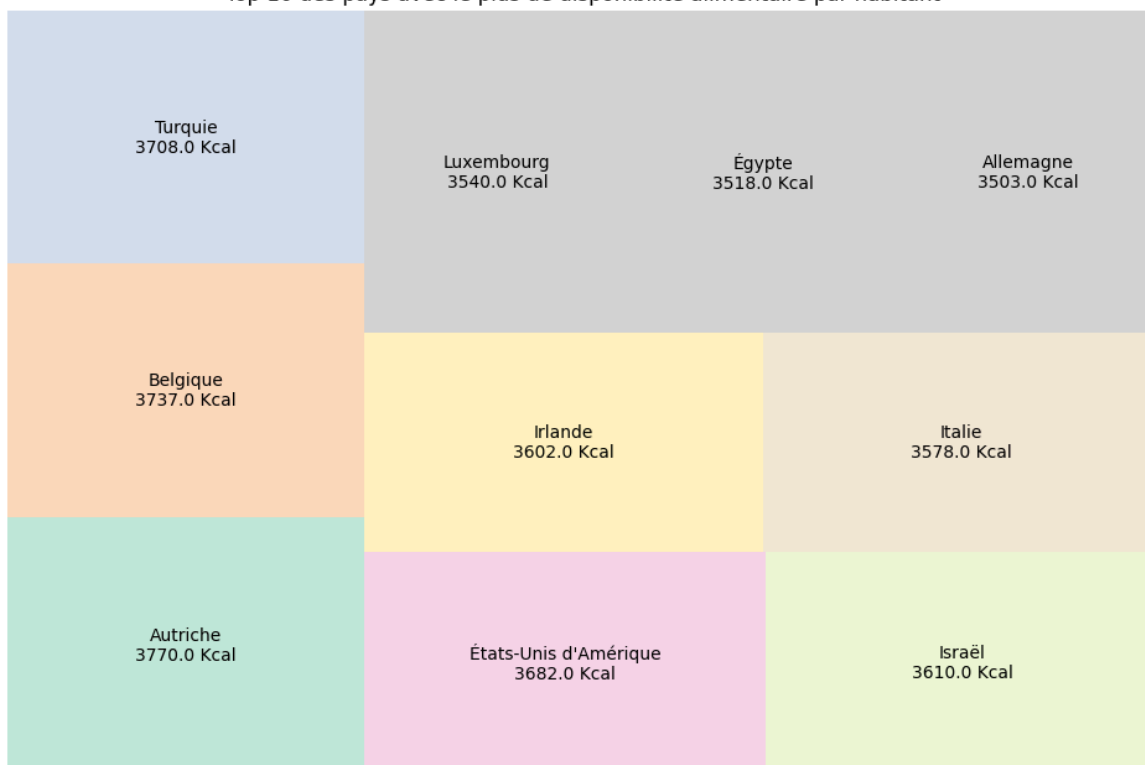
# Conversion des données en format numérique si nécessaire (suppression d
dispo_avec_pop['disponibilite_alimentaire_kcal_par_personne_par_jour'] =

# Trier les pays par disponibilité alimentaire décroissante et sélectionn
top_10_highest = dispo_avec_pop.sort_values(by='disponibilite_alimentaire

# Création d'un Tree Map
plt.figure(figsize=(12, 8))
squarify.plot(
    sizes=top_10_highest['disponibilite_alimentaire_kcal_par_personne_par
    label=top_10_highest['zone'] + "\n" + top_10_highest['disponibilite_a
    alpha=0.8,
    color=plt.cm.Pastel2(range(len(top_10_highest)))
)
plt.axis('off')
plt.title('Top 10 des pays avec le plus de disponibilité alimentaire par
plt.show()

```

Top 10 des pays avec le plus de disponibilité alimentaire par habitant



```
In [262... # Correction des colonnes avec conversion en valeurs numériques
dispo_avec_pop['pertes'] = dispo_avec_pop['pertes'].astype(str).str.repla
dispo_avec_pop['disponibilite_interieure'] = dispo_avec_pop['disponibilit

# Calcul de la proportion des pertes alimentaires
dispo_avec_pop['proportion_pertes'] = (dispo_avec_pop['pertes'] / dispo_a

# Calcul de la proportion totale
proportion_totale_pertes = round(dispo_avec_pop['proportion_pertes'].mean
print(f"La proportion moyenne des pertes alimentaires est de {proportion_
```

La proportion moyenne des pertes alimentaires est de 4.7%

```
In [265... # Liste des pays considérés comme riches (exemple)
pays_riches = ['États-Unis', 'France', 'Allemagne', 'Royaume-Uni', 'Japon

# Création de la colonne 'categorie_pays'
dispo_avec_pop['categorie_pays'] = dispo_avec_pop['zone'].apply(lambda x:
```

```
In [ ]: import matplotlib.pyplot as plt
import pandas as pd

# Création du DataFrame avec les données fournies
data = {
    'Pays': [
        'République Arabe Syrienne', 'Éthiopie', 'Yémen', 'Soudan du Sud'
        'Kenya', 'Bangladesh', 'Somalie', 'République Démocratique du Con
        'Haïti', 'République Populaire Démocratique de Corée', 'Madagasca
        'Lesotho', 'Tchad', 'Rwanda', 'Mozambique', 'Timor-Leste', 'Afgha
    ],
    'Aide_Alimentaire': [800, 700, 950, 600, 650, 500, 550, 750, 900, 580
    'Taux_Sous_Nutrition': [25, 20, 35, 30, 28, 15, 18, 32, 34, 27, 45, 4
}

# Conversion en DataFrame
```

```

df = pd.DataFrame(data)

# Création du scatter plot
plt.figure(figsize=(10, 6))
plt.scatter(df['Aide_Alimentaire'], df['Taux_Sous_Nutrition'], color='orange')

# Ajout des annotations pour chaque point
for i in range(len(df)):
    plt.text(df['Aide_Alimentaire'][i] + 10, df['Taux_Sous_Nutrition'][i],
             df['Pays'][i], fontsize=8, alpha=0.9)

# Paramètres du graphique
plt.xlabel('Aide Alimentaire Reçue (en milliers de tonnes)')
plt.ylabel('Taux de Sous-Nutrition (%)')
plt.title('Corrélation entre Aide Alimentaire et Taux de Sous-Nutrition')
plt.grid(True)
plt.tight_layout()

# Affichage du graphique
plt.show()

```

